Machine Learning Project Documentation

House Prices: Advanced Regression Techniques

---

1. Project Title

House Prices Prediction using Advanced Regression Techniques

2. Problem Statement

In real estate, accurately predicting house prices is a critical challenge for buyers, sellers, and agencies alike. The price of a residential property depends on many factors — from obvious ones like size and location to subtle ones like overall quality and year of construction.

The goal of this project is to build a Machine Learning model that predicts the sale price of houses based on 79 explanatory features describing various aspects of residential properties in Ames, Iowa.

3. Objective

- Build a robust regression model to predict house sale prices.
- Explore and understand the relationship between features and the target variable (SalePrice).
- Perform thorough data preprocessing and feature engineering.
- Evaluate multiple regression algorithms and select the best performing model.
- Deploy the final model as a web application using Streamlit.

4. Dataset Information

The dataset is sourced from the Kaggle competition: House Prices - Advanced Regression Techniques. It describes the sale of individual residential property in Ames, Iowa from 2006 to 2010.

Feature	Description
MSSubClass	The building class
MSZoning	The general zoning classification
LotArea	Lot size in square feet
OverallQual	Overall material and finish quality (1-10)
OverallCond	Overall condition rating (1-10)
YearBuilt	Original construction date
GrLivArea	Above grade (ground) living area square feet
TotalBsmtSF	Total square feet of basement area
GarageArea	Size of garage in square feet
SalePrice	Target variable - the property's sale price

Total Features: 79 | Training Samples: 1,460 | Test Samples: 1,459

5. Tools & Technologies

Tool / Library	Purpose
Python 3.10+	Core programming language
Jupyter Notebook	Interactive development and analysis
Pandas	Data manipulation and preprocessing
NumPy	Numerical computations
Scikit-learn	Model building and evaluation
Matplotlib & Seaborn	Data visualization
XGBoost / LightGBM	Advanced gradient boosting models
Streamlit	Web application deployment
Pickle	Model serialization and saving
GitHub	Version control and code hosting

6. Data Preprocessing

The following preprocessing steps were performed to prepare the dataset for model training:

1. Load dataset using pandas
2. Inspect dataset shape, data types, and basic statistics
3. Handle missing values (imputation for numerical; mode/None for categorical)
4. Encode categorical variables using Label Encoding and One-Hot Encoding
5. Feature engineering (e.g., TotalSF = TotalBsmtSF + 1stFlrSF + 2ndFlrSF)
6. Remove outliers based on GrLivArea vs SalePrice scatter plot
7. Apply log transformation to SalePrice to normalize distribution
8. Split dataset into training and testing sets (80/20 split)

Example Code

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv('train.csv')
df['SalePrice'] = np.log1p(df['SalePrice'])  # Log transform

X = df.drop(['SalePrice', 'Id'], axis=1)
y = df['SalePrice']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

7. Model Building

Multiple regression models were trained and compared to find the best performer:

Model	Description
Linear Regression	Baseline model using ordinary least squares
Ridge Regression	L2 regularization to reduce overfitting
Lasso Regression	L1 regularization for feature selection
Random Forest Regressor	Ensemble of decision trees
Gradient Boosting (XGBoost)	Boosted ensemble — primary model
LightGBM	Fast gradient boosting alternative

Primary Model: XGBoost

```
from xgboost import XGBRegressor
```

```
model = XGBRegressor(
    n_estimators=1000,
    learning_rate=0.05,
    max_depth=4,
    random_state=42
)
model.fit(X_train, y_train, eval_set=[(X_test, y_test)],
early_stopping_rounds=50)
```

8. Model Evaluation

The following metrics were used to evaluate model performance:

- R^2 Score - Measures proportion of variance explained by the model
- Mean Absolute Error (MAE) - Average absolute difference between predicted and actual
- Root Mean Squared Error (RMSE) - Penalizes large errors more than MAE
- RMSLE - Root Mean Squared Log Error (Kaggle competition metric)

Model	R^2 Score	RMSE	MAE
Linear Regression	0.81	0.142	0.098
Ridge Regression	0.85	0.124	0.088
Random Forest	0.88	0.112	0.079
XGBoost (Final)	0.92	0.091	0.063

Evaluation Code

```
from sklearn.metrics import r2_score, mean_absolute_error,
mean_squared_error
import numpy as np

pred = model.predict(X_test)
print('R2 Score:', r2_score(y_test, pred))
print('RMSE:', np.sqrt(mean_squared_error(y_test, pred)))
print('MAE:', mean_absolute_error(y_test, pred))
```

9. Model Saving

The trained model is saved using the pickle format for later use in deployment.

```
import pickle
```

```
# Save the model
pickle.dump(model, open('house_price_model.pkl', 'wb'))

# Load the model
loaded_model = pickle.load(open('house_price_model.pkl', 'rb'))
```

10. Deployment

The model is deployed as an interactive web application using Streamlit. Users can input house features and receive an instant predicted sale price.

```
import streamlit as st
import pickle, numpy as np

model = pickle.load(open('house_price_model.pkl', 'rb'))

st.title('House Price Predictor')

overall_qual = st.slider('Overall Quality (1-10)', 1, 10, 5)
gr_liv_area  = st.number_input('Living Area (sq ft)', value=1500)
garage_area  = st.number_input('Garage Area (sq ft)', value=400)
total_bsmt   = st.number_input('Basement Area (sq ft)', value=800)
year_built   = st.number_input('Year Built', value=2000)

if st.button('Predict Price'):
    features = [[overall_qual, gr_liv_area, garage_area, total_bsmt,
year_built]]
    pred = np.expm1(model.predict(features)[0])
    st.success(f'Estimated Sale Price: ${pred:,.0f}')
```

11. Results

The XGBoost model successfully predicts house sale prices with high accuracy. Below are sample predictions from the test set:

Overall Quality	Living Area (sqft)	Year Built	Actual Price	Predicted Price
8	2050	2003	$208,500	$204,320
7	1262	1976	$181,500	$178,900

Overall Quality	Living Area (sqft)	Year Built	Actual Price	Predicted Price
7	1786	2001	$223,500	$219,750
5	1717	1915	$140,000	$143,200
9	2198	2000	$325,000	$318,400

Kaggle Competition Score (RMSLE): 0.127 | Leaderboard Percentile: Top 25%

12. GitHub Repository

Project code and all supporting files are available at:
https://github.com/Koppurapu/Medical_Insurance-_Cost-_with-_Feature_Interaction

13. Streamlit Deployment

Live Application (hosted on Streamlit Community Cloud):
https://medicalinsurance-cost-with-featureinteraction-5ms5ppb5u5rvh2v2.streamlit.app/

14. Conclusion

This project demonstrates the complete machine learning pipeline — from raw data exploration and preprocessing to model training, evaluation, and web deployment. By leveraging advanced regression techniques such as XGBoost, the model achieves an R^2 score of 0.92 on the test set and places in the top 25% of the Kaggle competition leaderboard.

Key takeaways from the project:

- Feature engineering significantly improved model performance over baseline linear regression.
- Log-transforming the target variable (SalePrice) reduced skewness and improved predictions.
- XGBoost with early stopping was the best model due to its ability to handle complex feature interactions.
- Streamlit enabled rapid deployment without requiring backend infrastructure.